

Assignment-7

Q1) Write a program to implement Bankers algorithm considering processes P_0 to P_4 as shown in table and check if the system is in a safe state? If it is in safe state, determine the safe sequence.

```
#include <stdio.h>
```

```
int main() {
```

```
    int n=5, m=3, i, j, k;
```

```
    int allocation[5][3] = { {2,1,2}, {2,1,2}, {4,0,1},  
                               {0,2,0}, {1,1,2} };
```

```
    int max[5][3] = { {4,3,3}, {3,2,2}, {9,0,2}, {7,5,3},  
                       {1,1,2} };
```

```
    int available[3] = {2,1,0};
```

```
    int f[n], ans[n], ind=0;
```

```
    for(k=0; k<n; k++) {  
        f[k]=0;
```

```
    }
```

```
    int need[n][m];
```

```
    for(i=0; i<n; i++) {
```

```
        for(j=0; j<m; j++)
```

```
            need[i][j] = max[i][j] - allocation[i][j];
```

```
}
```

Name: Avancha Kiran Kumar
RegNo: 20233093

Name: Avancha Kirankumar
RegNo: 20233093

```
int y=0;
```

```
for (k=0; k<5; k++) {
```

```
    for (i=0; i<n; i++) {
```

```
        if (f[i] == 0) {
```

```
            int flag=0;
```

```
            for (j=0; j<m; j++) {
```

```
                if (need[i][j] > available[j]) {
```

```
                    flag=1;
```

```
                    break;
```

```
                }
```

```
            }
```

```
        }
```

```
    if (flag == 0) {
```

```
        ans[ind++] = i;
```

```
        for (y=0; y<m; y++) {
```

```
            available[y] += allocation[i][y];
```

```
        f[i] = 1;
```

```
    }
```

```
int flag=1;
```

```
for (int i=0; i<n; i++) {
```

```
    if (f[i] == 0) {
```

```
        flag=0;
```

```
        printf("The following system is not safe");
```

```
        break;
```

```
    }
```

```
}
```



```

if(flag==1){
    printf("System is in Safe Sequence\n");
    printf("Safe Sequence:");
    for(i=0; i<n-1; i++)
        printf("p%d -> ", ans[i]);
    printf("p%d\n", ans[n-1]);
}

```

```

}
return 0;

```

Name: Avamika Kirankumar
Reg No: 20233093

Q2) Write a multithreaded program that implements the baker's algorithm. Create n threads that request and release resources from the bank. The banker will grant the request only if it leaves the system in a safe state. You may write this program using μ threads.

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <stdlib.h>
#define NUM_PROCESSES 5
#define NUM_RESOURCES 3
int available[NUM_RESOURCES] = {2, 1, 0};
int max[NUM_PROCESSES][NUM_RESOURCES] =
    {{4, 3, 3}, {2, 2, 2}, {1, 0, 2}, {2, 5, 3}, {1, 1, 2}};

```



```
int allocation[NUM_PROCESSES][NUM_RESOURCES] =
{ {1,1,2}, {2,1,2}, {4,0,1}, {0,2,0}, {1,1,2} };
```

```
int need[NUM_PROCESSES][NUM_RESOURCES];
```

```
pthread_mutex_t lock;
```

```
void calculate_need() {
```

```
for (int i=0; i<NUM_PROCESSES; i++) {
```

```
for (int j=0; j<NUM_RESOURCES; j++) {
```

```
need[i][j] = max(i)[j] - allocation[i][j];
```

```
} }
```

Name: Avancha Kiran Kumar
RegNo: 2023 3093

```
int is_safe() {
```

```
int work[NUM_RESOURCES];
```

```
int finish[NUM_PROCESSES] = {0};
```

```
for (int i=0; i<NUM_RESOURCES; i++) {
```

```
work[i] = available[i];
```

```
}
```

```
int count = 0;
```

```
while (count < NUM_PROCESSES) {
```

```
int found = 0;
```

```
for (int i=0; i<NUM_PROCESSES; i++) {
```

```
if (finish[i] == 0) {
```

```
int can_allocate = 1;
```



```
for(int j=0; j < NUM_RESOURCES; j++) {
```

```
    if (need[i][j] > work[j]) {
```

```
        can_allocate = 0;
```

```
        break;
```

```
    }
```

```
    if (can_allocate) {
```

```
        for(int j=0; j < NUM_RESOURCES; j++) {
```

```
            work[j] += allocation[i][j];
```

```
        }
```

```
        finish[i] = 1;
```

```
        count++;
```

```
        found = 1;
```

```
    }
```

```
    }
```

```
    if (found == 0) {
```

```
        return 0;
```

```
    }
```

```
}
```

```
return 1;
```

Name: Avincha Kiran

RegNo: 20233053

```
int request_resources(int process_id, int request[]) {
```

```
    pthread_mutex_lock(&lock);
```

```
    for(int i=0; i < NUM_RESOURCES; i++) {
```

```
        if (request[i] > need[process_id][i]) {
```

```
            pthread_mutex_unlock(&lock);
```

```
            printf("p %d request exceeds its need\n", process_id);
```

```
            return 0;
```

```
    }
```



```
for(int i=0; i<NUM_RESOURCES; i++){
```

```
if(request[i] > available[i]){
```

```
pthread_mutex_unlock(&lock);
```

```
printf("p%d request exceeds available resources
```

```
Request denied.\n", process_id);
```

```
return 0;
```

```
}
```

```
for(int i=0; i<NUM_RESOURCES; i++){
```

```
available[i] = request[i];
```

```
allocation[process_id][i] += request[i];
```

```
need[process_id][i] -= request[i];
```

```
}
```

```
if(! is_safe()){
```

```
for(int i=0; i<NUM_RESOURCES; i++){
```

```
available[i] += request[i];
```

```
allocation[process_id][i] -= request[i];
```

```
need[process_id][i] += request[i];
```

```
pthread_mutex_unlock(&lock);
```

```
printf("p %d request leads to unsafe state .
```

```
Request denied.\n", process_id);
```

```
return 0;
```

```
}
```

Name: Avomcha Kiran

RegNo: 20233053


```

printf("P%d request granted.\n", process-id);
pthread_mutex_unlock(&lock);
return 1;

```

2

```

void release_resources(int process-id) {

```

```

    pthread_mutex_lock(&lock);

```

```

    for(int i=0; i<NUM_RESOURCES; i++) {

```

Name: Avancha Kiran

Regno: 20233093

```

        available[i] += allocation[process-id][i];

```

```

        allocation[process-id][i] = 0;

```

```

        need[process-id][i] = max[process-id][i];
    }

```

```

    printf("P%d releasing resources.\n", process-id);

```

```

    pthread_mutex_unlock(&lock);

```

```

void* process_thread(void* arg) {

```

```

    int process-id = *(int*)arg;

```

```

    int request[NUM_RESOURCES];

```

```

    for(int i=0; i<NUM_RESOURCES; i++) {

```

```

        request[i] = rand() % (need[process-id][i]);
    }

```

2

```

    printf("P%d requesting: [%d, %d, %d]\n",

```

```

        process-id, request[0], request[1], request[2]);

```

```

    if(request_resources(process-id, request)) {

```

```

        release_resources(process-id);
    }

```

2

```

    pthread_exit(NULL);

```


Name: Arancha K Franklun
RegNo: 20233093

```
int main() {
```

```
pthread_t threads[NUM_PROCESSES];
```

```
int process_ids[NUM_PROCESSES];
```

```
pthread_mutex_init(&lock, NULL);
```

```
calculate_need();
```

```
for(int i=0; i<NUM_PROCESSES; i++) {
```

```
    process_ids[i] = i;
```

```
    pthread_create(&threads[i], NULL, process_thread,  
                  &process_ids[i]);
```

```
}
```

```
for(int i=0; i<NUM_PROCESSES; i++) {
```

```
    pthread_join(threads[i], NULL);
```

```
printf("All processes completed successfully : \n");
```

```
pthread_mutex_destroy(&lock);
```

```
return 0;
```

```
}
```